

## **Lab Exercise 8 - Design and Develop a Web Crawler for Automated Web Content Discovery**

### **Objective**

To implement a web crawler using Spring Boot for automated discovery and retrieval of web content, manage the set of URLs to be crawled efficiently, avoid repeated crawling of the same pages, and analyze the performance of the crawler under different workloads.

### **Problem Statement**

A web content discovery system needs to automatically visit web pages, extract links from them, and discover additional pages for crawling. For example, when the crawler is provided with a seed URL such as <https://example.com>, it should retrieve the page, extract the hyperlinks present in the page, and add the newly discovered URLs to the crawl queue.

As the number of URLs and pages increases, maintaining the crawl queue and checking whether a URL has already been visited can affect the efficiency and scalability of the system. To improve the crawler, implement a queue-based crawling mechanism, maintain a visited-URL set to avoid duplicate crawling, and use Redis to manage or cache frequently accessed crawling information.

Develop the web crawler using Spring Boot, expose the crawling functionality through a REST API, and evaluate the system performance using JMeter.

### **Tasks**

- Create a dataset containing seed URLs and sample web pages or web resources to be crawled.
- Develop a Spring Boot application implementing the web crawling functionality.
- Implement a URL queue to maintain the URLs waiting to be crawled.
- Retrieve web pages from the URLs in the crawling queue.
- Extract hyperlinks from the retrieved web pages and add newly discovered URLs to the queue.
- Maintain a visited-URL set to ensure that the same URL is not crawled repeatedly.
- Implement appropriate validation to process only valid and permitted URLs.
- Store relevant crawling information such as discovered URLs, visited URLs, and crawl status.
- Integrate Redis to store or cache crawling state and frequently accessed URL information.

- Use the visited-URL information from Redis to avoid duplicate crawling when applicable.
- Expose the crawler functionality through a REST API and test it using Postman.
- Deploy the application and Redis using Docker.
- Use JMeter to generate concurrent crawling requests and analyze the performance of the web crawler.

## **Analysis**

Explain the role of:

- URL Queue
  - Seed URL
  - Visited-URL Set
  - Web Crawler
  - Redis Cache
1. Explain how the crawler discovers new URLs from a retrieved web page.
  2. Explain how the system determines whether a newly discovered URL should be crawled.
  3. Explain how the system processes a URL that has not been visited.
  4. Explain how the system processes a URL that has already been visited.
  5. Compare queue-based URL crawling with repeatedly scanning the complete collection of URLs.
  6. Analyze the performance of the crawler under different numbers of concurrent requests using JMeter.

## **Expected Learning Outcomes**

- Develop a web crawler using Spring Boot.
- Implement automated URL discovery and crawling using a URL queue.
- Manage visited URLs to prevent duplicate crawling.
- Integrate Redis for managing or caching crawling state.
- Expose and test web crawler functionality through REST APIs.
- Deploy the web crawler and Redis using Docker.
- Analyze the performance and scalability of the web crawler under concurrent workloads.